

Práctica 4: Node.js

Desarrollo de Sistemas Distribuidos

1. Introducción:

En esta práctica nos vamos a meter de lleno en el mundo de los sistemas distribuidos utilizando **Node.js**, **Socket.io** y **MongoDB**. El objetivo principal es comprender cómo podemos construir servicios web modernos para que sean eficientes, asíncronos y capaces de comunicarse e intercambiar información en tiempo real. En la parte final de esta práctica aplicaremos estos conocimientos para construir un sistema más complejo, un sistema domótico distribuido.

Para entenderlo que estamos haciendo, vamos a realizar un repaso de los conceptos teóricos clave en los que se apoya nuestro código:

- **El objetivo de Node.js:** Nos permite usar JavaScript en el servidor mediante un modelo por eventos. Está diseñado para maximizar la escalabilidad y eficiencia en servicios distribuidos.
- **Modelo Asíncrono y no bloqueante:** Se ejecuta todo el código en el mismo hilo, delegando las tareas pesadas de entrada/salida al SO. Esto evita que el programa se congele y le permite atender muchas peticiones simultáneas.
- **WebSockets:** Librería que establece un canal de comunicación bidireccional entre el navegador y servidor, rompiendo con el modelo HTTP tradicional de petición-respuesta para mantener una conexión abierta permanente.
- **Bases de datos NoSQL:** En vez de guardar la información en tablas cerradas y estructuradas, la guardan en documentos con un formato parecido al Json. Es ideal para sistemas distribuidos donde los datos cambian constantemente.
- **Sistemas de Módulos:** Es el estandar en JavaScript. Para que funcione bien tenemos que activar la directiva ‘ “type”:”module” ‘ en los archivos de configuración del proyecto.

2. Entorno de Desarrollo y Despliegue

Las instrucciones del guión plantean el desarrollo de manera general. El entorno en el que voy a desarrollarlo es una máquina Ubuntu 24.04 usando como Vs code como editor de código.

Trabajar con VS Code tiene muchas ventajas. Nos permite almacenar todas las dependencias en la raíz del proyecto y editar los archivos de configuración “package.json” sin necesidad de configurar de base IDEs de programación.

Además de esto, aprovechamos el hoisting de Node.js al instalar los paquetes de socket.io y mongodb en la raíz del proyecto, todas las subcarpetas heredan el acceso a las librerías de forma transparente.

Es por ello que la estructura del proyecto quedaría organizada de la siguiente manera:

```
Practica4_Node/
├── Ejemplo1/
│   └── package.json    (Módulos HTTP y servicios tipo REST)
├── Ejemplo2/
│   └── package.json    (Aplicaciones en tiempo real con Socket.io)
├── Ejemplo3/
│   └── package.json    (Persistencia y conectores de MongoDB)
├── Practica4_Domotico/
│   └── package.json    (El sistema domótico final extendido)
├── node_modules/      (Directorio raíz de librerías instaladas)
├── package.json        (Configuración global del espacio de trabajo)
└── package-lock.json   (Árbol de versiones de dependencias)
```

Compilación y ejecución:

- *Nota: ten cuidado de situarse en el directorio del ejercicio concreto que queramos ejecutar.*

1. Ejecutar el archivo principal del servidor usando “**node nombre_fichero.js**”
2. Abrir el navegador e introducir la dirección local para ver la ejecución (Ej: <http://localhost:8080>)
3. Al llegar al final del script, el programa no termina su ejecución si no que permanece activo a la escucha de puertos de comunicación que abramos o servicios sockets que usemos.

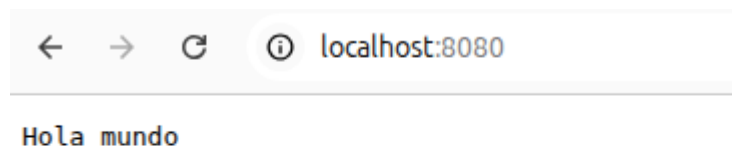
4. Implementación y análisis de los ejemplos

En esta primera fase del proyecto, he implementado y analizado los tres ejemplos que se proponen en el guión para comprender el comportamiento interno de cómo Node.js es capaz de levantar servidores web y gestionar peticiones mediante el protocolo HTTP.

- 4.1 Servicio HTTP Básico (helloworld.js):

Este script simplemente tiene la estructura básica donde utilizamos el método `createServer` del módulo `HTTP`. A este método le pasamos los métodos `request` (la petición del cliente) y el `response` (la respuesta que se construye en el código). Aquí el servidor simplemente atiende cualquier petición del puerto 8080 devolviendo un código 200, que es un estándar en sistemas para indicar que todo está OK. Además de el texto plano "Hola mundo "

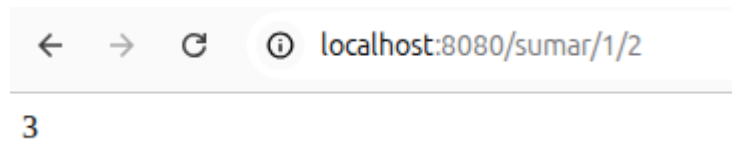
- Ejecutamos: `node helloworld.js`
- Visitamos: `http://localhost:8080/`



- 4.2. Calculadora REST Distribuida (calculadora.js)

En este ejemplo implementamos un servicio tipo REST. Ahora el servidor debe inspeccionar la URL revisando el parámetro `request`. Con funciones de strings, extraemos los 3 parámetros claves, la operación el primer valor y el segundo valor. El código además valida mediante condicionales que los parámetros sean correctos. Después de esto simplemente se calcula el resultado y enviamos en formato HTML a diferencia del anterior que se enviaba en texto plano.

- **Ejecutamos:** node calculadora.js
- **Visitamos:** <http://localhost:8080/sumar/1/2>



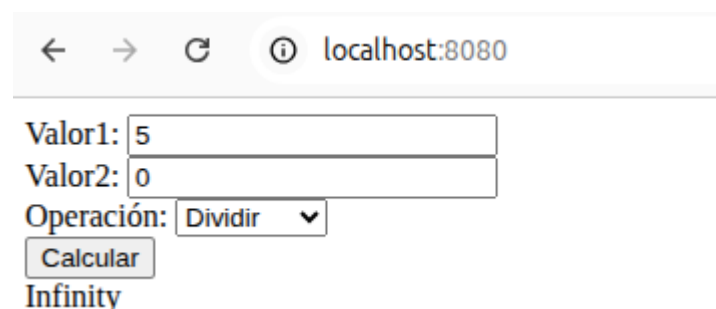
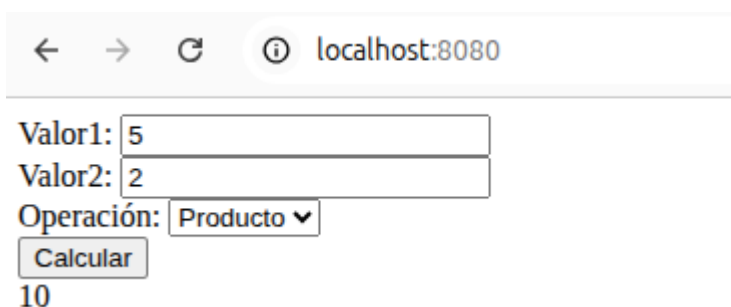
4.3. Servidor Web Completo con Interfaz (calculadora-web.js y calc.html)

En este último ejemplo unimos backend con frontend. Además tenemos que importar las librerías path y fs para construir rutas e interactuar con el FileSystem respectivamente. Ahora nuestro servidor puede recibir información de dos formas distintas.

1. Igual que antes con una petición REST.
2. Si detecta que le pasamos una url vacía (exactamente la raíz /), el servidor lee los parámetros que se introducen dentro del formulario de calc.html de manera asíncrona mediante readFile() y se lo envía al navegador.

Además es importante recalcar que dentro del html, se utiliza JavaScript para interceptar el envío del formulario. Mediante fetch envía los datos construyendo la url, recibiendo el resultado de forma asíncrona e inyectándolo en la web sin necesidad de recargar la página completa.

- **Ejecutamos:** node calculadora-web.js
- **Visitamos:** <http://localhost:8080/>



4.4. Aplicaciones en Tiempo Real con Socket.io (connections.js y connections.html)

En este ejemplo introducimos la librería de Socket.io que nos permite utilizar los WebSockets para establecer el canal de comunicación bidireccional entre el cliente y el servidor.

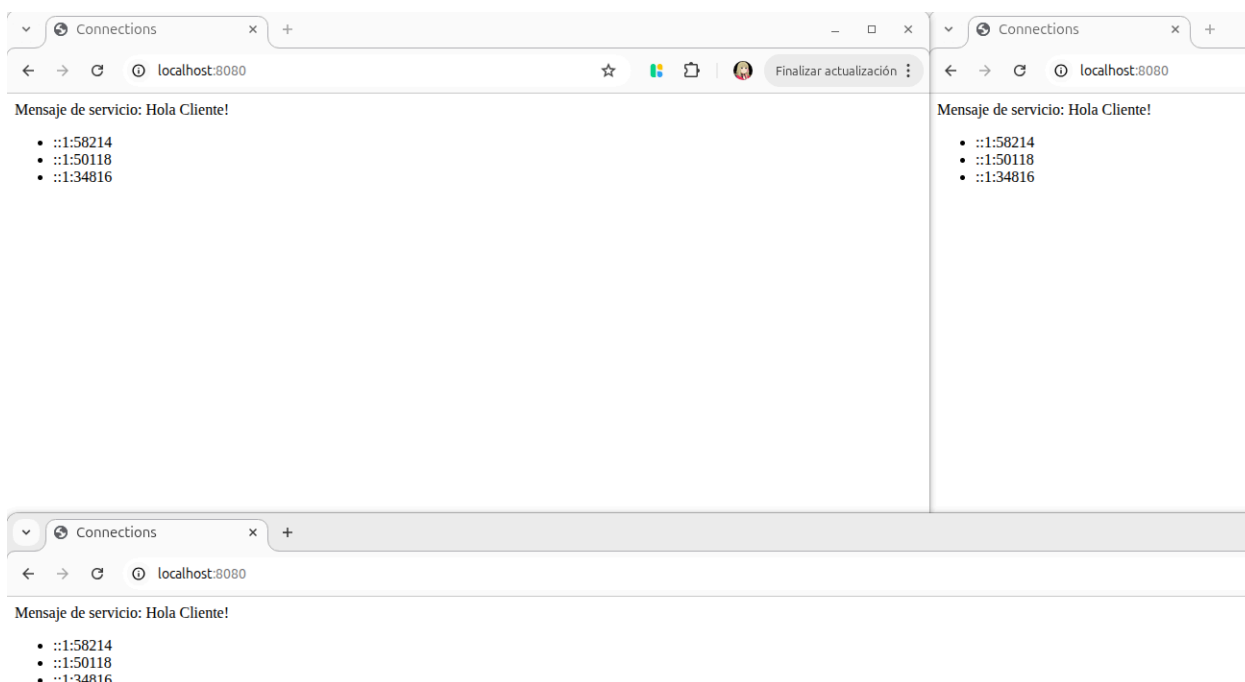
En el código el servidor (connections.js) mantiene un array en memoria con las direcciones IP y los puertos de los clientes activos. Cada vez que un usuario se conecta o se desconecta se actualiza esta estructura y se actualiza la nueva lista simultáneamente en el frontend gracias a la instrucción `io.socket.emit`.

Por otra parte, el cliente (connections.html) se suscribe a estos eventos. Cuando recibe la señal del servidor, utiliza código en JavaScript para manipular el DOM y redibujar la lista de usuarios en tiempo real, sin recargar la página en ningún momento. Además se añade un evento de desconexión para alertar al usuario cuando el servidor se cierra.

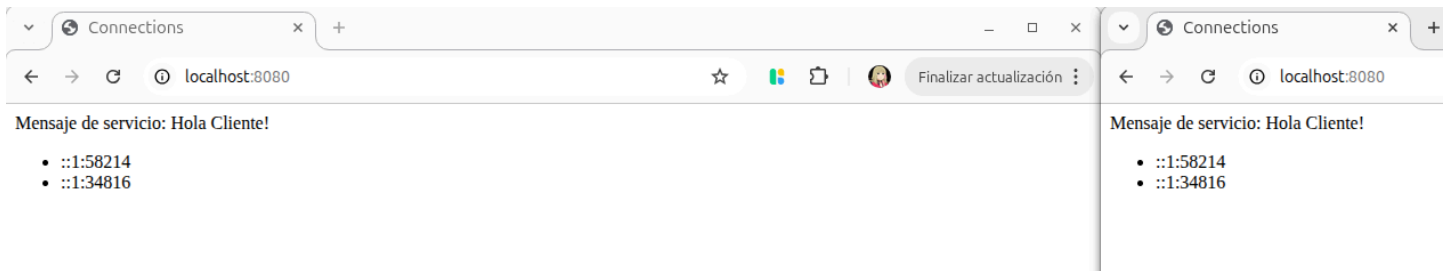
- **Ejecutamos:** `node connections.js`
- **Visitamos:** `http://localhost:8080/`

Para demostrar el correcto funcionamiento de la comunicación en tiempo real, se han obtenido los siguientes estados:

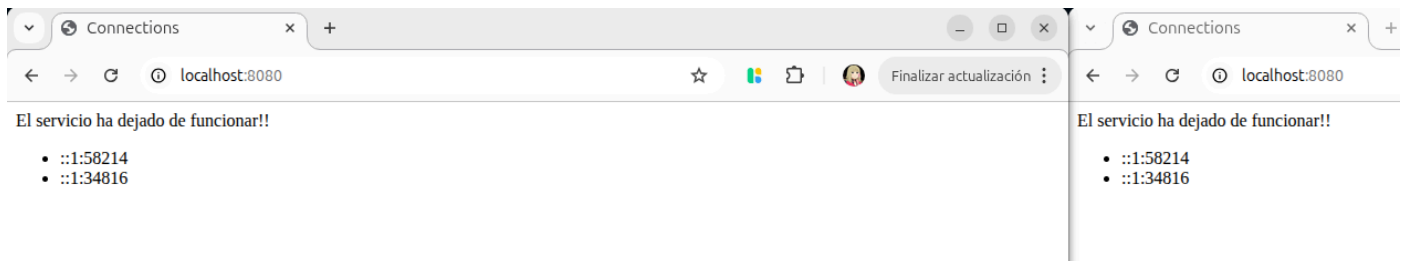
- **1º Estado:** Se abren tres ventanas de navegador distintas. Se observa como uno a uno el servidor detecta las conexiones y automáticamente actualiza la interfaz de las tres.



- **2º Estado:** Cuando se cierra una de las ventanas del navegador, el servidor lo detecta y se elimina ese usuario del array interno, actualizando a su vez la interfaz de todos los usuarios.



- **3º Estado:** Forzamos el cierre del servidor con un Ctrl+C en la terminal. Los clientes detectan automáticamente el evento de desconexión y se muestra el mensaje que los avisa por pantalla.



4.5. Uso de MongoDB desde Node.js (mongo-test.js y mongo-test.html)

En este último ejemplo, lo que hacemos es integrar una conexión de nuestro servidor Node.js con una base de datos MongoDB (NoSQL)

El servidor (mongo-test.js) establece primero la conexión con la base de datos utilizando MongoClient. Una vez conectado levanta el servicio HTTP y las comunicaciones WebSockets. EL código se basa en dos eventos clave, el que inserta un nuevo registro en la base de datos usando insertOne(), y el que devuelve todos los registros como si fuese un get.

Por otra parte, el cliente (mongo-test.html) establece la conexión, emite datos como su dirección, puerto y hora actual para que el servidor los almacene. Justo despues se solicita el historial completo para redibujarlo en el DOM.

Nota: Para que la ejecución funcione, hemos tenido que añadir dos líneas al principio del .js. Esto es por que en la versión en la que estamos trabajando v18 el módulo de criptografía hay que exponerlo manualmente.

- **Ejecutamos:** node mongo-test.js
- **Visitamos:** <http://localhost:8080/>

Para demostrar que la información persiste, se ha recargado la pestaña varias veces. En la siguiente captura se ve perfectamente que los datos no se reinician, si no que la base de datos los almacena permanente mostrando cómo se acumulan los distintos registros con identificadores y tiempos únicos.

5. Sistema Domótico Distribuido

5.1. Arquitectura del proyecto

Para el desarrollo del sistema domótico se ha diseñado una arquitectura centralizando toda la lógica de control en un único archivo (servidor.js) que coordina todo el flujo de datos entre el almacenamiento en la base de datos y las dos interfaces.

Para que funcione correctamente he tenido que instalar y configurar los siguientes módulos propios de Node a través del gestor de paquetes npm.

- **socket.io:** Se encarga de implementar los Websockets para comunicar bidireccionalmente el frontend del backend.
- **mongodb:** Es el driver oficial para la conexión con la base de datos NoSQL.
- **node-telegram-bot-api:** Biblioteca para interactuar con los mensajes automáticos de Telegram.

Todo el estilo del frontend para no mezclarlo con la estructura estática del html ni la lógica de programación del javascript lo hemos puesto aparte (public/estilos.css). No hemos querido recargar de estilos las páginas, así que hemos usado uno bastante sencillo, intentando reciclar las clases para ambos htmls, con variables generales para los colores y el uso de grid y flex para ordenar los elementos.

De esta manera la estructura final de proyecto obtenida a partir del comando tree queda de la siguiente manera:

```
— node_modules
  |— (...)
  |— package.json
  |— package-lock.json
  |— public
    |— estilos.css
    |— sensores.html
    |— usuario.html
  — servidor.js
```

5.2. Lógica del agente e implementación de otros eventos

El código actúa de forma inteligente evaluando constantemente los eventos entrantes que recogemos de los sensores. Para cumplir con lo que se pide en el guión hemos seguido las siguientes reglas:

- **Control de luminosidad:** Si el valor que se recoge de los sensores supera el 80%, el agente convierte el estado de la persiana a Cerrada. Por el contrario, si la luminosidad baja del 20%, el estado pasará a Abierta.
- **Control de temperatura:** Si la temperatura obtenida supera el umbral de los 26º, el sistema activa inmediatamente el Aire Acondicionado. A su vez si este baja de los 20º se desactiva.

Añadimos el evento “Derroche de energía”

Como funcionalidad aparte para mejorar la calificación de la práctica se han desarrollado más casuísticas añadiendo el elemento de “la ventana”. Esta es controlada manualmente por el usuario.

Cada vez que se detecta que el Aire Acondicionado se encuentra encendido al mismo tiempo que la ventana, se genera una alerta en la interfaz que avisa de este derroche. Si el estado de cualquiera de ambos elementos cambia rompiendo con esta coincidencia, el aviso desaparece automáticamente notificando los estados normales de consumo

5.3 Integración con APIs externas.

Se ha implementado dos servicios externos de internet:

- **API de Clima (Open-Meteo):** Durante la inicialización del servidor se realiza una petición HTTP fetch con los parámetros de las coordenadas geográficas de Granada. Este nos devuelve un Json con información meteorológica del tiempo en ese punto. El enlace es el siguiente:

https://api.open-meteo.com/v1/forecast?latitude=37.1882&longitude=-3.6067¤t_weather=true

De esta información recogemos el valor de la temperatura con el cual inicializamos el servidor cada vez que lo lanzamos.

- **API de Telegram:** esta API una vez que hemos configurado los bots y obtenido las keys a través de “BotFather”, está encargada de ir enviando notificaciones de forma instantánea al móvil cuando el agente detecta el evento en el que se encuentra la ventana y el Aire Acondicionado abierto al mismo tiempo. Obviamente por motivos de seguridad no he incluido mis credenciales personales, sin embargo para comprobar el funcionamiento el código está preparado para cargar el valor de estas (token y chatId), se encuentran en la línea 75 y 76.

```
//Integración con Api Telegram
// Configuración directa de Telegram (;ACUÉRDATE DE BORRAR LOS NÚMEROS AL ENTREGAR!)
const token = 'introduce aquí el token del bot';
const chatId = 'introduce aquí el id del caht';
let bot = new TelegramBot(token, {polling: false});
console.log("Bot de Telegram configurado y listo para avisar.");
```

5.4 Base de Datos y WebSockets:

Todas las acciones de la infraestructura (variaciones de los sensores, cambios del agente, comandos manuales del usuario o las alertas de la API de Telegram) se asocian con una marca de tiempo (new Date()) y se almacena en una base de datos llamada “sistemaDomotico” dentro de la colección de MOngoDB. Todas estas tuplas además de insertarse también se van mostrando en la interfaz

La comunicación en tiempo real y la consistencia de los datos en todas las pantallas abiertas simultáneamente se resuelve gracias al Socket io.

Al abrir una nueva interfaz de usuario, el servidor realiza una consulta ordenada a la base de datos y le transmite el histórico completo de todos los eventos de forma inmediata, permitiendo ver la lista dinámica en un mensaje global dentro de la interfaz.

Además, cada vez que se inserta una tupla de forma exitosa en MongoDB, el backend emite un mensaje global hacia todos los sockets. El JavaScript del navegador coje este paquete y se encarga de actualizar la interfaz haciendo que los registros aparezcan instantáneamente en la parte superior del historial del usuario.

6. Guía de uso y Demostración

6.1. Instrucciones para el despliegue

Para iniciar y ver la solución que se ha propuesto, se deben seguir los siguientes pasos dentro de la terminal de Linux:

1. Asegurarse de tener el servicio de MongoDB activo.
2. Estar situaciones en el directorio del proyecto (Practica4_Domotico)
3. Lanzar el script del backend con “node [servidor.js](#)”
4. Ver gracias a los logs iniciales por terminal que se ha inicializado todo bien.
5. Abrir en dos pestañas distintas de navegador los siguientes enlaces:

```
zapi24@zapi24-GF63-Thin-95C:~/Escritorio/3 2ºCuatri/DSD/Practicas-DSD/P4/Practica4_Domotico$ node servidor.js
Bot de Telegram configurado y listo para avisar.
Conexion exitosa a MongoDB
Consultando la temperatura de Granada...
=====
SISTEMA DOMÓTICO INICIADO
Panel de usuario: http://localhost:8080/
Panel de sensores: http://localhost:8080/sensores
=====
```

6.2. Prueba de la ejecución:

A continuación se insertan varias capturas de la ejecución del código, donde se ven distintos estados mostrando así todos los requisitos implementados:

